

Natural Language Processing

Sesión II

Rafael Sánchez, Ph.D.
contacto@rafaelsanchez.dev

Transfer
learning

BERT

NLP Tasks

1

NLP - Sesión II

01 Transfer Learning en NLP. Hubs de modelos. Librerías

02 Pre-training

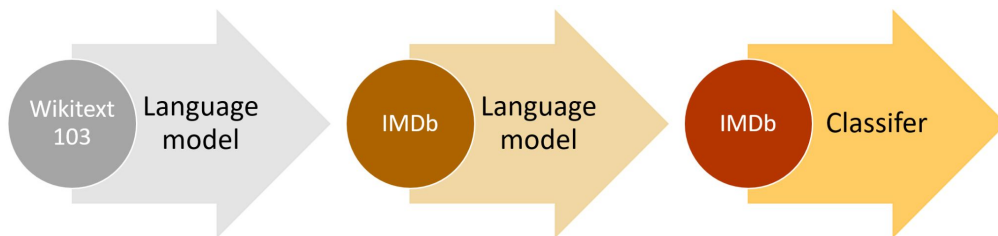
03 BERT. BERT pre-training

04 T5

05 NLP tasks. Use cases for finance with transformers

Transfer Learning en NLP

- Transfer Learning es una técnica usada para imágenes que entrena un modelo (pej: ResNet) en una tarea, y luego la adapta (fine-tuning) a otra tarea con un número de datos reducido.
- Aunque funciona bien en aplicaciones de imágenes, durante años no se ha podido replicar en aplicaciones de lenguaje natural (NLP). Típicamente se necesitaban grandes cantidades de datos y lo hacía poco viable.
- En 2018, finalmente, se propuso un framework para adaptar modelos pre-entrenados .



Fuente: <https://nlp.fast.ai/classification/2018/05/15/introducing-ulmfit.html>

Kaggle models

- As of November 15th, 2023, models in TensorFlow Hub are moved to Kaggle, and tfhub.dev domain and UI will be deprecated.
- The www.kaggle.com/models repository provides many pre-trained models: text embeddings, image classification models, TF.js/TFLite models and much more.
- The repository is open to community contributors. It still supports old tfhub.dev links.

```
!pip install --upgrade tensorflow_hub
import tensorflow_hub as hub
model = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim128/2")
embeddings = model(["The rain in Spain.", "falls", "mainly", "In the plain!"])
print(embeddings.shape) # (4,128)
```

Kaggle models and TensorFlow Hub

- Only tfhub.dev UI is deprecated. The TensorFlow Hub repository still exist containing 2,300+ TensorFlow models published by Google DeepMind.
- `tensorflow_hub` python package will still exist
- `kagglehub` python package allows to download and upload to kaggle models

```
import kagglehub

# Download the latest version.
kagglehub.model_download('google/bert/tensorFlow2/answer-equivalence-bem')

# Download a specific version.
kagglehub.model_download('google/bert/tensorFlow2/answer-equivalence-bem/1')

# Download a single file.
kagglehub.model_download('google/bert/tensorFlow2/answer-equivalence-bem',
path='variables/variables.index')
```

Datasets and models in Colab

- Datasets from Kaggle
- Models from Kaggle like Gemma
- Kaggle competitions
- Shows info and code to easily download with `kagglehub`

The screenshot displays the Kaggle interface for the 'Credit Card Fraud Detection' dataset. It includes a sidebar with navigation icons, a main content area with the dataset title and description, a code snippet for downloading the dataset, and a summary of the dataset's structure.

About this dataset

Credit Card Fraud Detection

Machine Learning Group - ULB • 13,452 upvotes • 8 y ago

Context It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that th...

[Read more](#)

Code snippet

```
import kagglehub
path = kagglehub.dataset_download("mlg-ulb/creditcardfraud")
```

Dataset

- creditcardfraud

Summary

- 1 file
 - .csv 1
- 31 columns
 - Decimal 29
 - Integer 2

Hugging Face Hub

- Hugging Face Hub is a platform with over 2M models and 1,5M datasets, and 1,5M demo apps (called “Spaces”), all open source and publicly available, in an online platform where people can easily collaborate and build ML together.

```
from huggingface_hub import login
login()
# Alternatively, from terminal, huggingface-cli login
```

```
from huggingface_hub import hf_hub_download
hf_hub_download(repo_id="google/pegasus-xsum", filename="config.json")
```

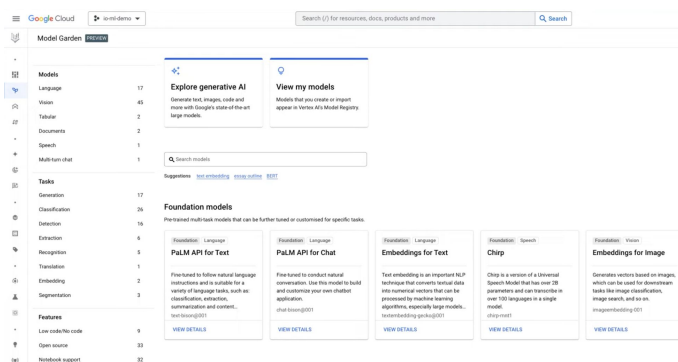
TensorFlow Model garden

- Collection of models **TensorFlow/Keras only** (no Pytorch or other frameworks).
- Include models about language understanding (BERT), image classification (EfficientNet), object detection (YOLO), instance segmentation, video classification (MoViNets) and more.
- Trainable on:
 - Distributed training on multiple GPUs
 - Distributed training on multiple GPU hosts
 - Distributed training on Cloud TPUs
- <https://github.com/tensorflow/models/tree/master/official>

```
pip3 install tf-models-official==2.8.0
pip3 install tensorflow-text==2.8.0 # when models in uses `nlp` packages
```

Google Cloud Platform Model garden

- Proprietary hub by Google Cloud providing a **curated collection** of 130+ models including enterprise-ready foundation model APIs, open source models, and task-specific models from Google and 3rd parties.



Pre-trained BERT

BERT, as many other models, is available and ready to run fine-tuning. Available in models hubs and its own page:

<https://github.com/google-research/bert#pre-trained-models>

- BERT Base:** 12 layers, 768-dimensional hidden representations, 12 attention heads in each self-attention module, and 110M parameters.
- BERT Large:** 24 layers, 1024-dimensional hidden representations, 16 attention heads in each self-attention module, and 340M parameters.

Keras 3 - New multi backend

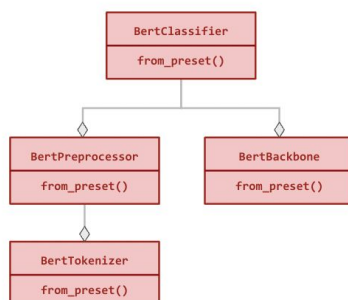
K

- **Full rewrite** of Keras library that supports JAX, TensorFlow and Pytorch
- `keras.distribution` library: large-scale model parallelism & data parallelism with JAX
- Data pipelines from any source: `fit()` / `evaluate()` / `predict()` compatible with `tf.data.Dataset` objects, PyTorch `DataLoader`, NumPy arrays and Pandas dataframes.
- `keras.ops` library: custom layers, custom losses, custom metrics and custom optimizers will work with JAX, PyTorch, and TensorFlow.
- LLM capabilities: Keras NLP and Keras CV built on top of Keras 3

Keras 3 - High level API overview

K

```
classifier = keras_nlp.models.BertClassifier.from_preset("bert_tiny_en_uncased")
backbone = keras_nlp.models.BertBackbone.from_preset("bert_tiny_en_uncased")
tokenizer = keras_nlp.models.BertTokenizer.from_preset("bert_tiny_en_uncased")
preprocessor = keras_nlp.models.BertPreprocessor.from_preset("bert_tiny_en_uncased")
```



keras_nlp.models.XXTokenizer



```
# Custom vocabulary.
vocab = ["[UNK]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"]
vocab += ["The", "quick", "brown", "fox", "jumped", "."]

tokenizer = keras_nlp.models.BertTokenizer(vocabulary=vocab)
preprocessor = keras_nlp.models.BertPreprocessor(tokenizer)

preprocessor("The quick brown fox jumped.")
```

Tokenizer: `keras_nlp.models.XXTokenizer`

What it does: Converts strings to sequences of token ids.

Why it's important: The raw bytes of a string are too high dimensional to be useful features so we first map them to a small number of tokens, for example "The quick brown fox" to ["the", "qu", "##ick", "br", "##own", "fox"].

Inherits from: `keras.layers.Layer`.

keras_nlp.models.XXPreprocessor



```
preprocessor = keras_nlp.models.BertPreprocessor.from_preset(
    "bert_base_en_uncased"
)

# Tokenize and pack a single sentence.
preprocessor("The quick brown fox jumped.")

# Tokenize a batch of single sentences.
preprocessor(["The quick brown fox jumped.", "Call me Ishmael."])
```

Preprocessor: `keras_nlp.models.XXPreprocessor`

What it does: Converts strings to a dictionary of preprocessed tensors consumed by the backbone, starting with tokenization.

Why it's important: Each model uses special tokens and extra tensors to understand the input such as delimiting input segments and identifying padding tokens. Padding each sequence to the same length improves computational efficiency.

Has a: `XXTokenizer`.

Inherits from: `keras.layers.Layer`.

keras_nlp.models.XXBackbone

```
input_data = tf.ones(shape=(8, 224, 224, 3))

# Pretrained backbone
model = keras_cv.models.ResNetBackbone.from_preset("resnet50_imagenet")
output = model(input_data)

# Randomly initialized backbone with a custom config
model = ResNetBackbone(
    stackwise_filters=[64, 128, 256, 512],
    stackwise_blocks=[2, 2, 2, 2],
    stackwise_strides=[1, 2, 2, 2],
    include_rescaling=False,
)
output = model(input_data)
```

Backbone: `keras_nlp.models.XXBackbone`

What it does: Converts preprocessed tensors to dense features. Does not handle strings; call the preprocessor first.

Why it's important: The backbone distills the input tokens into dense features that can be used in downstream tasks. It is generally pretrained on a language modeling task using massive amounts of unlabeled data. Transferring this information to a new task is a major breakthrough in modern NLP.

Inherits from: `keras.Model`.

keras_nlp.models.XXClassifier

```
classifier = keras_nlp.models.BertClassifier.from_preset("bert_tiny_en_uncased_sst2")

# Note: batched inputs expected so must wrap string in iterable
classifier.predict(["I love modular workflows in keras-nlp!"])
classifier.evaluate(imdb_test)
```

Task: e.g., `keras_nlp.models.XXClassifier`

What it does: Converts strings to task-specific output (e.g., classification probabilities).

Why it's important: Task models combine string preprocessing and the backbone model with task-specific Layers to solve a problem such as sentence classification, token classification, or text generation. The additional Layers must be fine-tuned on labeled data.

Has a: `XXBackbone` and `XXPreprocessor`.

Inherits from: `keras.Model`.

Hugging Face transformers library



- SDK to work with a wide range of transformer models as well as tools to adapt these models to new use cases.
- Supports **three major deep learning frameworks**: TensorFlow, Pytorch and Jax.
- Provides task-specific heads so you can easily finetune transformers on downstream tasks such as text classification and others.

Hugging Face AutoTokenizers library



- Fast tokenizer library, written in Rust, different from the tokenizers written in Python part of the transformers library.
- Allows to retrain a model from scratch when adapting the model to a different language or different corpus.
- By default, `AutoTokenizer` tries to load a fast tokenizer if it's available, otherwise, it loads the Python implementation.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
example   = "I like Artificial Intelligence"
encoding  = tokenizer(example)

encoding.tokens()
encoding.word_ids()
```

Hugging Face datasets library



- Preprocess the data with `Dataset.map`
- Load and compute metrics
- Slicing
- Pandas adaptation

```
from datasets import load_dataset

data_files = {"train": "drugsComTrain_raw.tsv", "test": "drugsComTest_raw.tsv"}
drug_dataset = load_dataset("csv", data_files=data_files, delimiter="\t")
drug_sample = drug_dataset["train"].shuffle(seed=42).select(range(1000))

# Peek at the first few examples
drug_sample[:3]
```

Auto classes



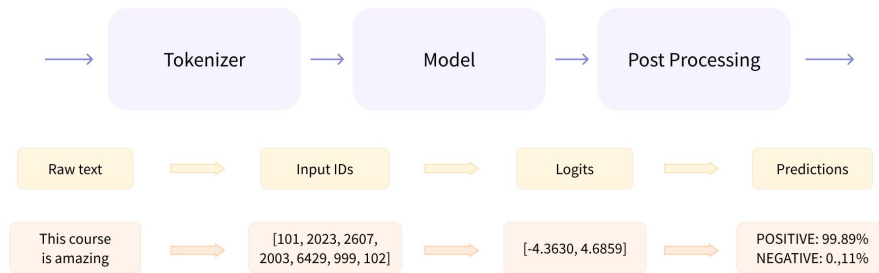
- In many cases, the architecture you want to use can be guessed from the name or the path of the pretrained model you are supplying to the `.from_pretrained()` method.
- AutoClasses automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary.
- Examples:
 - `AutoModelForCausalLM` for causal language models
 - `AutoModelForMaskedLM` for masked language models
 - `AutoModelForSeq2SeqLM` for encoder-decoder models

```
model = AutoModel.from_pretrained("bert-base-cased")
model = AutoModelForSequenceClassification.from_pretrained("ProsusAI/finbert")
```

transformers.pipeline



- Pipeline groups together three steps:
 - Preprocessing
 - Passing the inputs through the model
 - Postprocessing



Chat templates



- Different models may use different formats or control tokens for the chat.
- Chat templates indicate how chats must be properly formatted, by using the `apply_chat_template` method.
- The input to `apply_chat_template` should be structured as a **list of dictionaries** with `role` and `content` keys. The `role` key specifies the speaker, and the `content` key contains the message. The common roles are:
 - `user` for messages from the user
 - `assistant` for messages from the model
 - `system` for directives on how the model should act (usually placed at the beginning of the chat)

Chat templates: example for Mistral



```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-v0.1")
chat = [
    {"role": "user", "content": "Hello, how are you?"},
    {"role": "assistant", "content": "I'm doing great. How can I help you today?"},
    {"role": "user", "content": "I'd like to show off how chat templating works!"},
]

tokenizer.apply_chat_template(chat, tokenize=False)
# <s>[INST] Hello, how are you? [/INST]I'm doing great. How can I help you today?</s>
[INST] I'd like to show off how chat templating works! [/INST]
```

evaluate library



- Library to evaluate machine learning models and datasets.
- Replaces `metrics` library (deprecated).
- Example of metrics: Accuracy, Exact Match, Mean Intersection over union (Mean IoU, for images), others.

```
python -c "import evaluate;

print(evaluate.load('exact_match').compute(references=['hello'], predictions=['hello']))"
# {'exact_match': 1.0}
```

Demo Hugging Face Spaces



- <https://huggingface.co/spaces/Swindu/ProsusAI-finbert>
- <https://huggingface.co/spaces/huggingface-projects/gemma-3-12b-it>
- <https://huggingface.co/spaces/deepseek-ai/deepseek-vl2-small>
- <https://huggingface.co/spaces/Qwen/Qwen2.5-72B-Instruct>

NLP - Sesión II

01 Transfer Learning en NLP. Hubs de modelos

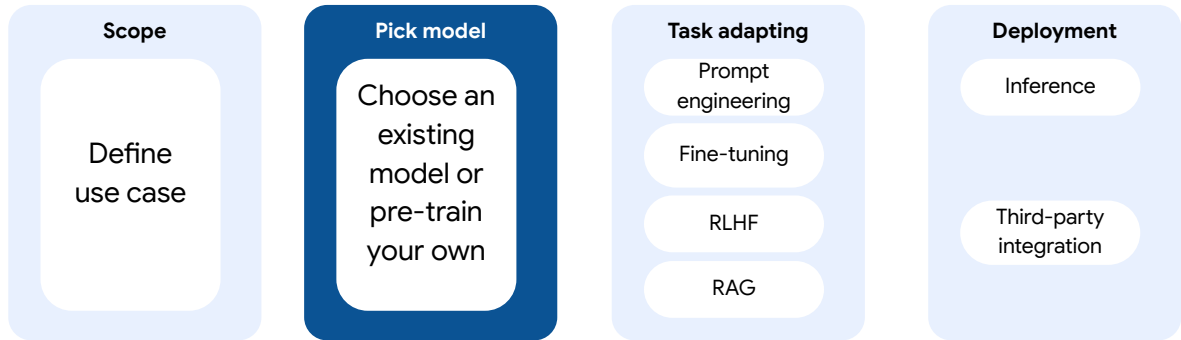
02 Pre-training

03 BERT. BERT pre-training

04 T5

05 NLP tasks. Use cases for finance with transformers

LLM model lifecycle



Pre-training

Foundational model

Pretrained LLM

Pre-train your own model

Custom LLM

Foundation models

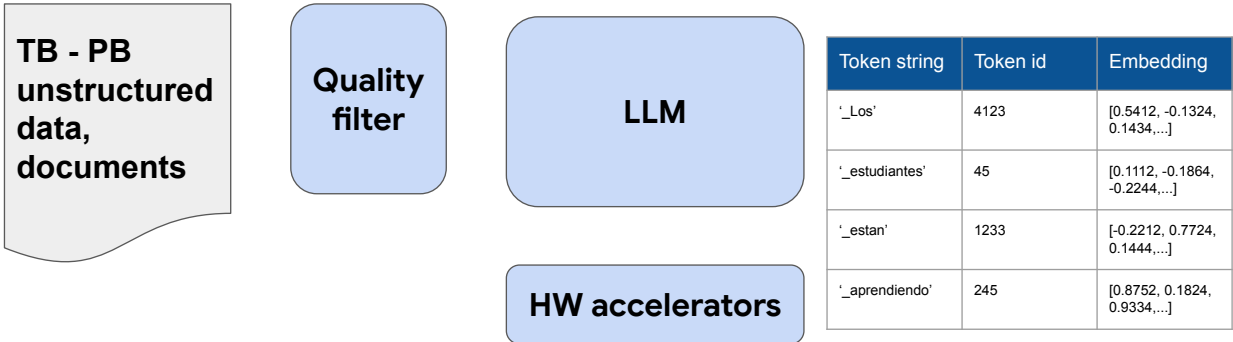
Foundation models are models trained on broad data (generally using self-supervision at scale) that can be adapted to a wide range of downstream tasks. Foundation models are a **superset of LLMs**, although most foundation models are LLMs (e.g., BERT, T5, GPT-3).

Source: <https://hai.stanford.edu/news/reflections-foundation-models>

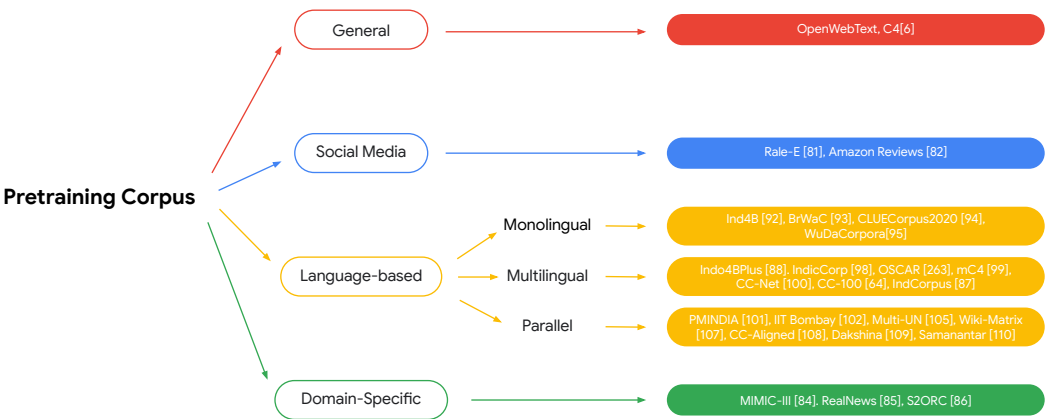
Example of foundation models

- BERT
- T5
- PaLM, Gemini
- Llama
- Mistral
- Many others

LLM pre-training at high-level



Datasets used to pre-train LLMs



Datasets used to pre-train LLMs

GPT-3

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

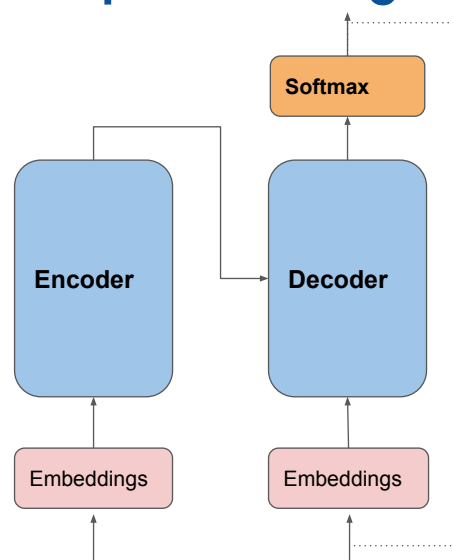
PaLM

Data source	Proportion of data
Social media conversations (multilingual)	50%
Filtered webpages (multilingual)	27%
Books (English)	13%
GitHub (code)	5%
Wikipedia (multilingual)	4%
News (English)	1%

Total dataset size = 780 billion tokens

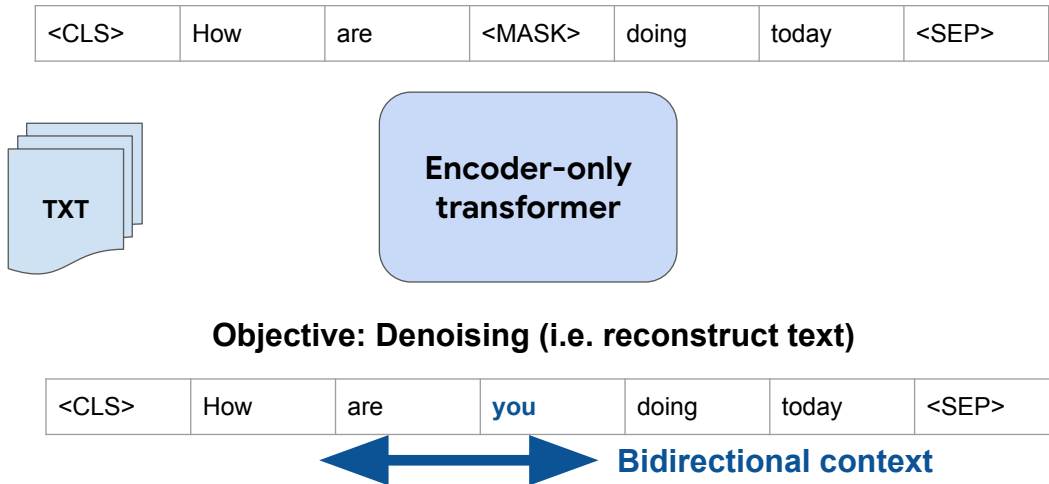
Transformer architectures for pre-training

- Encoder-only
- Decoder-only
- Encoder-decoder



Encoder-only transformers

Masked Language Model (MLM)

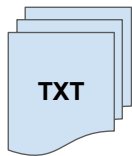


Encoder-only transformers

- Also known as **autoencoders**
- Example of encoder-only transformers:
 - BERT: bidirectional encoder
 - ALBERT: a Light BERT
 - LaBSE
 - RoBERTa: which has been trained for far more steps than the original BERT.
 - FinBERT
- Use cases for encoders-only architectures:
 - Classification, including sentiment, i.e. a text is negative or positive.
 - Entity recognition

Decoder-only transformers

Causal Language Modeling (CLM)



How	are	?
-----	-----	---



Objective: Predict next token

How	are	you
-----	-----	-----

→ **Unidirectional context**

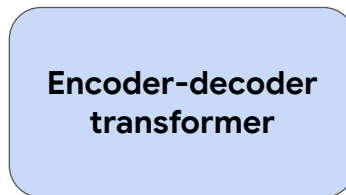
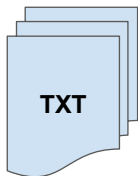
Decoder-only transformers

- Also known as **autoregressive models**.
- Examples of decoder-only transformers:
 - Meena2/Lamda: open-ended generation
 - GPT-3: open-ended generation
- Use cases for decoders-only transformers:
 - Open-ended text generation, like generating sequences, i.e., chatbots

Encoder-decoder transformers

Span Corruption

How	are	<MASK>	<MASK>	today
How	are	<X>		today



<X>: Sentinel token

Objective: Reconstruct span

<X>	you	doing
-----	-----	-------

Encoder-decoder transformers

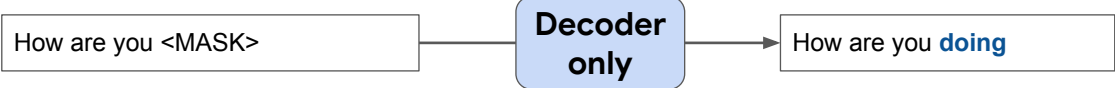
- Examples of encoder-decoder transformers
 - T5, mT5: Bidirectional encoder with unidirectional decoder
- Use cases:
 - Constrained generation (abstractive summarization)
 - Translation
 - Can be used for many fine-tuning applications: classification, Q&A, translation, summarization, sentiment

Summary of pre-training objectives

Autoencoding: denoising



Autoregressive: predict next token

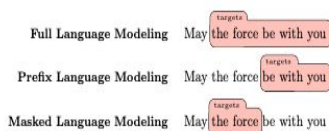


Seq2seq: reconstruct span



Choosing the pre-training objective

- The choice of pretraining objective can have **significant impact on the downstream usability of the LLM**.
 - Masked language modeling: for autoencoders
 - Language modeling: for causal decoder-only models
 - Prefix language modeling and Span corruption: for encoder-decoder and non-causal decoder-only models.
- There can be many training objectives.



Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BEFT-style Devlin et al. (2018)	Thank you <D> <D> me to your party apple week .	(original text)
Deshauling	party me for your to . last fun you inviting week . Thank	(original text)
MASS-style Song et al. (2019)	Thank you <D> <D> me to your party <D> week .	(original text)
L.i.d. noise, replace spans	Thank you <D> me to your party <Y> week .	<D> for inviting <Y> last <D>
L.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <D> to <Y> week .	<D> for inviting me <Y> your party last <D>

Language Model Architecture and Pretraining Objective [Wang2022-wa]

Masked LMs and its variants in T5
T5 [Raffel2019-kc]

NLP - Sesión II

01 Transfer Learning en NLP. Hubs de modelos

02 Pre-training

03 BERT. BERT pre-training

04 T5

05 NLP tasks. Use cases for finance with transformers

Bidirectional Encoder Representations from Transformers (BERT)

- **Pre-training sin supervisar y bidireccional:**
 - Esto significa que el modelo se entrena en un corpus de texto grande (como Wikipedia) y luego usa ese modelo para tareas posteriores de NLP.
 - Sin supervisión significa que BERT se entrenó utilizando solo un corpus de texto: implementa la técnica de **Masked Language Modeling (MLM)** que enmascara el 15% de todos los tokens de entrada de forma aleatoria.
 - Usa **Next Sentence Prediction (NSP)** para relacionar frases.
- Las representaciones pre-entrenadas también pueden ser contextuales o libres de contexto, y las representaciones contextuales pueden ser además unidireccionales o bidireccionales:
 - Por ejemplo, en la frase "*Hice un depósito bancario*", una representación unidireccional de "*banco*" solo se basa en "*Hice*", pero no en "*depósito*".
- Arquitectura similar a la de los transformers estándar, pero con 12 capas y 12 heads de atención. El BERT básico ("BERT base") tiene 110M parámetros, el grande ("BERT large") 340M.
- Reduce los recursos necesarios para el fine-tuning: 30 minutos con una sola TPU o pocas horas con una sola GPU.
- Open source en <https://github.com/google-research/bert>

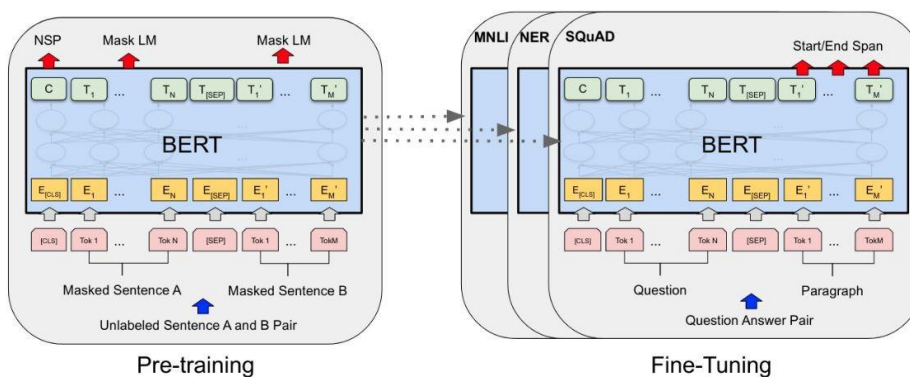
[BERT \[Devlin2018-ca\]](#)

BERT: pre-entrenamiento y fine-tuning

- BERT aprovecha un enfoque basado en el fine-tuning para aplicar modelos lingüísticos pre-entrenados; es decir, se entrena para una tarea relativamente genérica y, a continuación, se ajusta en tareas posteriores específicas que pueden ser más o menos similares a la tarea de pre-entrenamiento.
- Tareas durante el pre-entrenamiento:
 - Masked Language Modeling (MLM)
 - Next Sentence Prediction (NSP)

[BERT \[Devlin2018-ca\]](#)

BERT: pre-entrenamiento y fine-tuning



[BERT \[Devlin2018-ca\]](#)

Tokens y Embeddings en BERT

- Tokens especiales
 - **[CLS]**: token de clasificación. Se utiliza al principio de una secuencia.
 - **[SEP]**: indica la separación de 2 secuencias, es decir, actúa como delimitador.
 - **[MASK]**: indica el token enmascarado en la tarea MLM.
- **Segment Embeddings**: se usan para indicar la secuencia a la que pertenece un token. es decir, si hay varias secuencias separadas por un token [SEP] en la entrada, junto con las información posicional, se añaden estos segment embeddings para que el modelo identifique la secuencia del token.
- Los tokens que se introducen en el modelo BERT se tokenizan utilizando embeddings con el tokenizer WordPiece. Básicamente es una técnica de tokenización entre la codificación puramente a nivel de caracteres y la codificación completa a nivel de palabras para aumentar la cobertura de la mayoría de las palabras en el vocabulario con una reducción apreciable en el tamaño del vocabulario (es decir, la mayoría de las palabras fuera del vocabulario o OOV están cubiertas).

[BERT \[Devlin2018-ca\]](#)

Tokens y Embeddings en BERT

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	#ing	[SEP]
Token Embeddings	E _[CLS]	E _{my}	E _{dog}	E _{is}	E _{cute}	E _[SEP]	E _{he}	E _{likes}	E _{play}	E _{#ing}	E _[SEP]
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E _A	E _A	E _A	E _A	E _A	E _A	E _B	E _B	E _B	E _B	E _B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E ₀	E ₁	E ₂	E ₃	E ₄	E ₅	E ₆	E ₇	E ₈	E ₉	E ₁₀

[BERT \[Devlin2018-ca\]](#)

BERT: tarea MLM

- En esta tarea, el **15% de los tokens de cada secuencia se enmascaran aleatoriamente** (se sustituyen por el token `[MASK]`). La función de pérdida sólo aplica sobre los token `[MASK]`. El modelo se entrena para estimar estos tokens utilizando todos los demás tokens de la secuencia.
- Sin embargo, eso tiene un problema, ya que el modelo sólo aprende a estimar bien cuando vea el token `[MASK]`, no en el resto de palabras.
- La tarea de fine-tuning no verá nunca el token `[MASK]` en su entrada. Por tanto, para que el modelo se adapte a estos casos: el 80% de las veces los tokens del 15% se enmascaran; el 10% de las veces los tokens del 15% se sustituyen por tokens aleatorios; y el 10% de las veces se mantienen tal cual, es decir, sin tocar.
 - 80% of the time: Replace the word with the `[MASK]` token, e.g., my dog is hairy → my dog is `[MASK]`
 - 10% of the time: Replace the word with a random word, e.g., my dog is hairy → my dog is apple
 - 10% of the time: Keep the word unchanged, e.g., my dog is hairy → my dog is hairy. The purpose of this is to bias the representation towards the actual observed word.

[BERT \[Devlin2018-ca\]](#)

BERT: tarea NSP

- El 50% de las veces se utiliza correctamente la siguiente frase, y el 50% de las veces se toma una frase aleatoria del corpus para el entrenamiento. Esto garantiza que el modelo se adapte al entrenamiento en múltiples secuencias (para tareas como la respuesta a preguntas y la inferencia del lenguaje natural).

Input = `[CLS] the man went to [MASK] store [SEP]`
 `he bought a gallon [MASK] milk [SEP]`

Label = `IsNext`

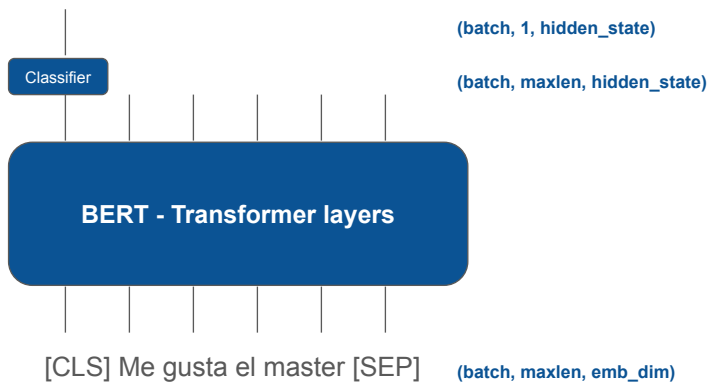
Input = `[CLS] the man [MASK] to the store [SEP]`
 `penguin [MASK] are flight ##less birds [SEP]`

Label = `NotNext`

[BERT \[Devlin2018-ca\]](#)

BERT - Fine-tuning para clasificación

- Como los transformers trabajan con contextos, la salida del token [CLS] habrá capturado información del resto de la frase (el contexto).
- La salida se introduce en una capa densa (clasificador) con tantas salidas como clases.



NLP - Sesión II

01 Transfer Learning en NLP. Hubs de modelos

02 Pre-training

03 BERT. BERT pre-training

04 T5

05 NLP tasks. Use cases for finance with transformers

Text-To-Text Transfer Transformer (T5)

- **Text-to-text encoder-decoder**, entrenado con el dataset C4 (doble de tamaño que la wikipedia).
- **mT5** es la variante multilinguaje entrenada con el dataset **mC4**.
- **Fine-tunable**: único modelo para múltiples tareas: clasificación, Q&A, traducción, resumen, y análisis de sentimiento.
- Se puede tunear usando el framework **T5X**.
- Arquitectura similar al transformer de Vaswani (12 etapas de encoder-decoder), con diferentes dimensiones de las matrices “key”, “value” and “query”.

Fuente: <https://ai.googleblog.com/2020/02/exploring-transfer-learning-with-t5.html>

[T5 \[Raffel2019-kc\]](#)

Text-To-Text Transfer Transformer (T5)

- Tamaños disponibles y descargas:
 - **T5-Small** (60 millones de parámetros): gs://t5-data/pretrained_models/small
 - **T5-Base** (220 millones de parámetros): gs://t5-data/pretrained_models/base
 - **T5-Large** (770 millones de parámetros): gs://t5-data/pretrained_models/large
 - **T5-3B** (3 billones de parámetros): gs://t5-data/pretrained_models/3B
 - **T5-11B** (11 billones de parámetros): gs://t5-data/pretrained_models/11B
- Más pequeño que GPT-3:
 - Menos recursos y coste durante el pre-entrenamiento y fine-tuning.
- Nótese que 11B de parámetros no caben en un acelerador con 16 GB de memoria, con lo que hay que partir el modelo, con frameworks como T5X (“*mesh tensorflow*”).

[T5 \[Raffel2019-kc\]](#)

Arquitectura del modelo T5-small

- Pre-training: dataset C4 de 34B tokens
 - Menos que BERT (137B tokens) ó RoBERTa (2.2T tokens).
- 12 heads
- 12 capas
- Dimensión de embeddings: **768**
- Dimensión de la capa densa: **3072**
- Arquitectura encoder-decoder
- Tokenizador **SentencePiece** y vocabulario de **32,000** tokens
- Total: **220 millones de parámetros** (aproximadamente el doble de BERT-base)

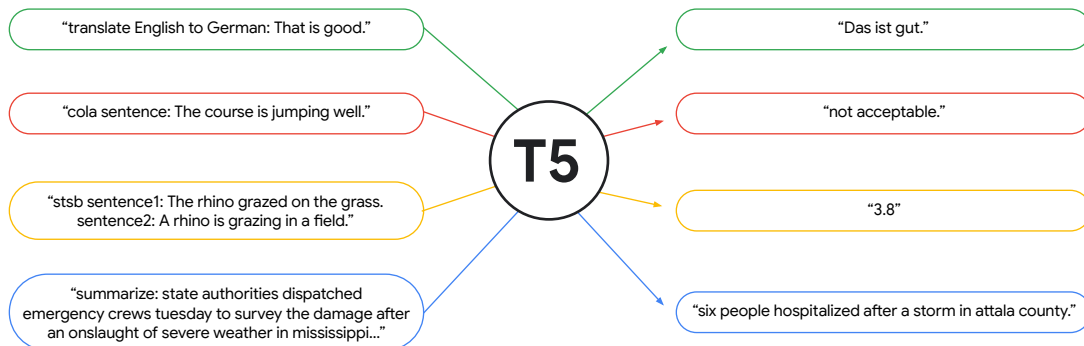
[T5 \[Raffel2019-kc\]](#)

Pre-training. Infra

- Combinación de paralelismo de datos y modelos, a través de la librería Mesh TensorFlow.
- Entrenamiento sobre “slices” de Cloud TPU Pods
 - 1,024 TPU v3 chips conectados por una red de alta velocidad 2D mesh



Datasets for fine-tuning



**Todos los datasets son convertidos en tareas “text-to-text”.
T5 fue pionero en el uso de instrucciones humanas**

[T5 \[Raffel2019-kc\]](#)

T5 - Data preprocessing CoLA



D.1 CoLA

Original input:

Sentence: John made Bill master of himself.

Processed input: cola sentence: John made Bill master of himself.

Original target: 1

Processed target: acceptable

[T5 \[Raffel2019-kc\]](#)

T5 - Data preprocessing MNLI



Original input:

Hypothesis: The St. Louis Cardinals have always won.

Premise: yeah well losing is 1 mean i'm i'm originally from Saint Louis and Saint Louis Cardinals when they were there were uh a mostly a losing team but

Processed input:

mnli hypothesis: The St. Louis Cardinals have always won. premise:
yeah well losing is 1 mean i'm i'm originally from Saint Louis and Saint Louis
Cardinals when they were there were uh a mostly a losing team but

Original target:

2

[T5 \[Raffel2019-kc\]](#)

T5 - Data preprocessing SQuAD



Original input:

Question: What does increased oxygen concentrations in the patient's lungs displace?

Context: Hyperbaric (high-pressure) medicine uses special oxygen chambers to increase the partial pressure of O₂ around the patient and, when needed, the medical staff. Carbon monoxide poisoning, gas gangrene, and decompression sickness (the 'bends') are sometimes treated using these devices. Increased O₂ concentration in the lungs helps to displace carbon monoxide from the heme group of hemoglobin. Oxygen gas is poisonous to the anaerobic bacteria that cause gas gangrene, so increasing its partial pressure helps kill them. Decompression sickness occurs in divers who decompress too quickly after a dive, resulting in bubbles of inert gas, mostly nitrogen and helium, forming in their blood. Increasing the pressure of O₂ as soon as possible is part of the treatment.

Processed input: question: What does increased oxygen concentrations in the patient's lungs displace? context: Hyperbaric (high-pressure) medicine uses special oxygen chambers to increase the partial pressure of O₂ around the patient and, when needed, the medical staff. Carbon monoxide poisoning, gas gangrene, and decompression sickness (the 'bends') are sometimes treated using these devices. Increased O₂ concentration in the lungs helps to displace carbon monoxide from the heme group of hemoglobin. Oxygen gas is poisonous to the anaerobic bacteria that cause gas gangrene, so increasing its partial pressure helps kill them. Decompression sickness occurs in divers who decompress too quickly after a dive, resulting in bubbles of inert gas, mostly nitrogen and helium, forming in their blood. Increasing the pressure of O₂ as soon as possible is part of the treatment.

Original target: carbon monoxide

Processed target: carbon monoxide

[T5 \[Raffel2019-kc\]](#)

Evaluación

- Evaluación de **T5 Small, Base, Large, 3 y 11B** contra el modelo “state-of-the-art” para varios benchmarks (para la fecha de 24 de Octubre de 2019).
- Todos los resultados sobre el set de test excepto SQuAD sobre el de validación.

Model	GLUE Average	CoLA Matthew's Accuracy	SST-2 Accuracy	MRPC F1	MRPC Accuracy	STS-B Pearson	STS-B Spearman
Previous best	89.4 ^a	69.2 ^b	97.1 ^c	93.6^d	91.5^e	92.7 ^f	92.3 ^g
T5-Small	77.4	41.0	91.8	89.7	86.6	85.6	85.0
T5-Base	82.7	51.1	95.2	90.7	87.5	89.4	88.6
T5-Large	86.4	61.2	96.3	92.4	89.9	89.9	89.2
T5-3B	88.5	67.1	97.4	92.5	90.0	90.6	89.8
T5-11B	90.3	71.6	97.5	92.8	90.4	93.1	92.8

Model	QQP F1	QQP Accuracy	MNLI-m Accuracy	MNLI-mm Accuracy	QNLI Accuracy	RTE Accuracy	WNLI Accuracy
Previous best	74.8 ^a	90.7^b	91.3 ^c	91.0 ^a	99.2^d	89.2 ^e	91.8^f
T5-Small	70.0	88.0	82.4	82.3	90.3	69.9	69.2
T5-Base	72.6	89.4	87.1	86.2	93.7	80.1	78.8
T5-Large	73.9	89.9	89.9	89.6	94.8	87.2	85.6
T5-3B	74.4	89.7	91.4	91.2	96.3	91.1	89.7
T5-11B	75.1	90.6	92.2	91.9	96.9	92.8	94.5

Model	SQuAD EM	SQuAD F1	SuperGLUE Average	BoolQ Accuracy	CB F1	CB Accuracy	COPA Accuracy
Previous best	90.1 ^a	95.5^b	84.6 ^c	87.1 ^d	90.5 ^e	95.2 ^f	90.6 ^g
T5-Small	79.10	87.24	63.3	76.4	54.9	81.6	46.0
T5-Base	85.44	92.08	76.2	81.4	86.2	94.0	71.2
T5-Large	86.66	93.79	82.3	85.4	91.6	94.8	83.4
T5-3B	88.53	94.95	86.4	89.9	90.3	94.4	92.0
T5-11B	91.26	96.22	88.9	91.2	93.9	96.8	94.8

Model	MultiRC F1a	MultiRC EM	ReCoRD F1	ReCoRD Accuracy	RTE Accuracy	WIC Accuracy	WSC Accuracy
Previous best	84.4 ^a	52.5 ^d	90.6 ^d	90.0 ^d	88.2 ^e	69.9 ^f	89.0 ^f
T5-Small	69.3	26.3	56.3	55.4	73.3	66.9	70.5
T5-Base	79.7	43.1	75.0	74.2	81.5	68.3	80.8
T5-Large	83.3	50.7	86.8	85.9	87.8	69.3	86.3
T5-3B	86.8	58.3	91.2	90.4	90.7	72.1	90.4
T5-11B	88.1	63.3	94.1	93.4	92.5	76.9	93.8

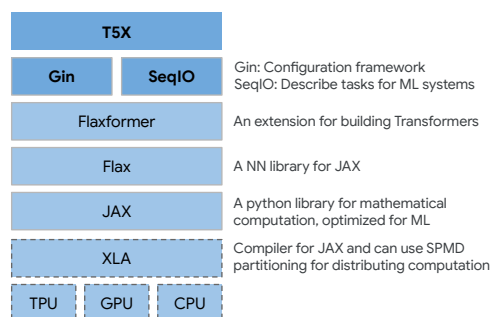
Model	WMT EnDe BLEU	WMT EnFr BLEU	WMT EnRo BLEU	CNN/DM ROUGE-1	CNN/DM ROUGE-2	CNN/DM ROUGE-L
Previous best	33.8^a	43.8^a	38.5^f	43.47 ^g	20.30 ^h	40.63 ⁱ
T5-Small	26.7	36.0	26.8	41.12	19.56	38.35
T5-Base	30.9	41.2	28.0	42.05	20.34	39.40
T5-Large	32.0	41.5	28.1	42.50	20.68	39.75
T5-3B	31.8	42.6	28.2	42.72	21.02	39.94
T5-11B	32.1	43.4	28.1	43.52	21.55	40.69

mT5

- Variante multilinguaje entrenada con el **dataset mC4**.
- mC4 es la variante multilinguaje del dataset C4: soporte de 100 lenguajes
- La arquitectura del modelo es similar a la de T5, en concreto a la variante T5.1.1, que mejora T5 usando **activaciones GeGLU** y aumenta dimensiones de Embeddings para algunas variantes (XL y XXL).
- Variantes mT5-Small, mT5-Base, mT5-Large, mT5-XL y mT5-XXL (sustituyen a 3B y 11B de T5).

T5X - Librería Open source basada en JAX

- T5X es una **librería basada en JAX**:
 - T5X es la librería usada por muchos modelos LLM de Google research como PaLM y UL2.
 - T5X se puede ejecutar en Vertex AI y permite el uso de Cloud TPUs.
- **Use cases (jobs)** of T5X:
 - Ejecución de un Fine-tuning
 - Ejecución de una evaluación
 - Ejecución de una inferencia batch
 - Pre-entrenar un modelo
- Open source:
<https://github.com/google-research/t5x>



SeqIO

- SeqIO is a library for **processing sequential data** to be fed into downstream sequence models.
 - Refactor of the `t5.data` library to train the T5 model
- Data sources can be: `TfdsDataSource` (TensorFlow Datasets); `TextLine DataSource` (tsv files); `TFExampleDataSource` (`tf.train.Example` protos from TFRecord files); `FunctionDataSource` for providing a custom function that returns a `tf.data.Dataset`.
- The most important class in SeqIO is the Task. It is an abstraction that combines:
 - a **raw data source**
 - one or more **preprocessing steps**
 - a **vocabulary** to tokenize/detokenize each preprocessed feature for the model
 - a **postprocessor** to convert detokenized model outputs into a format for evaluation
 - one or more **metrics** to evaluate with
- Open-source at <https://github.com/google/seqio>

Creating a SeqIO task

```
TaskRegistry.add(
    "wmt19_ende",
    source=seqio.TfdsDataSource(tfds_name="xsum:1.1.0"),
    preprocessors=[
        functools.partial(
            preprocessors.summarize,
            article_key="document",
            summary_key="summary"),
        seqio.preprocessors.tokenize,
        seqio.CacheDatasetPlaceholder(),
        functools.partial(
            prepend_prompt,
            prompt_mode=prompt_mode,
            mode="_prompt"),
        seqio.preprocessors.append_eos_after_trim,
    ],
    metric_fns=[metrics.rouge],
    output_features=DEFAULT_OUTPUT_FEATURES)
```

Gin

- **Configuration framework** for Python, based on dependency injection.
 - Avoids the need to define and maintain configuration objects (e.g. protos).
- Gin is well suited for machine learning experiments (e.g. using TensorFlow), which tend to have **many parameters**, often nested in complex ways.
- Open-source at <https://github.com/google/gin-config>
- Example for fine-tuning using the wmt_t2t dataset (english-german): `finetune_t511_base_wmt.gin`

```
from __gin__ import dynamic_registration

import __main__ as train_script
from t5.data import mixtures
from t5x import models
from t5x import partitioning
from t5x import utils

include "t5x/examples/t5/t5_1_1/base.gin"
include "t5x/configs/runs/finetune.gin"

MIXTURE_OR_TASK_NAME = "wmt_t2t_ende_v003"
TASK_FEATURE_LENGTHS = {"inputs": 256, "targets": 256}
TRAIN_STEPS = 1_003_000 # 1000000 pre-trained steps + 3000 fine-tuning steps.
DROPOUT_RATE = 0.0
INITIAL_CHECKPOINT_PATH = "gs://t5-data/pretrained_models/t5x/t5_1_1_base/checkpoint_1000000"
LOSS_NORMALIZING_FACTOR = 233472
EVAL_PERIOD = 500

BATCH_SIZE = 128
```

flaxformer

- **Configuration framework** for Python, based on dependency injection.
 - Avoids the need to define and maintain configuration objects (e.g. protos).
- Transformer library for primarily NLP and multimodal research at Google.
- Flaxformer is primarily used **together with T5X**. Standalone use less frequent.
- It is used for many NLP research use cases, providing both off-the-shelf BERT and T5 models, and several research projects built on shared components.
- Open-source at <https://github.com/google/flaxformer>

Why do we need T5X ?

- Parameters of Large Language models (LLMs) do not fit on one device - e.g. a 5-billion parameter language model.
 - T5X supports both data and model parallelism
- LLMs achieve major qualitative advantages by increasing model sizes and scaling up the size of training datasets.
 - T5X multiscales, and supports both data and model parallelism
- Operationalizing LLM is not trivial due to various factors including scale, performance and ensuring reproducible results.
 - T5X allows to use **Cloud TPU** to simplify the process to build and train LLMs at scale.

NLP - Sesión II

01 Transfer Learning en NLP. Hubs de modelos

02 Pre-training

03 BERT. BERT pre-training

04 T5

05 NLP tasks. Use cases for finance with transformers

Caso de uso: Resúmenes

- Un resumen es una técnica para crear una **versión reducida**, con toda la información relevante, de un texto ó un documento.
- Tipos de resúmenes:
 - **Extractivo**: extrae (copia exacta) partes concretas del documento.
 - **Abstracto**: genera texto nuevo que contiene la información más relevante.
- Modelos: GPT-2, T5, BART, PEGASUS

Datasets para resúmenes abstractos

- **XSum**: 226k noticias (inglés)
- **XLSum**: 1.35 millones de noticias en 44 lenguajes, incluido el español
- **BillSum**: 23k documentos de leyes del estado de california (inglés)
- **ArXiv**: 1.7M+ papers científicos, con título y su “abstract”.

Ejemplo de un registro del dataset XLSum en español

```
{  
  "id": "noticias-internacional-39424071",  
  "url":  
    "https://www.bbc.com/mundo/noticias-internacional-39424071",  
  "title": "Estados Unidos: 3 claves para entender la orden  
ejecutiva firmada por Donald Trump para dismantelar la política  
medioambiental de Barack Obama",  
  "summary": "\"Vamos a poner a nuestros mineros a trabajar otra  
vez\", dijo el presidente Donald Trump este martes al firmar una  
nueva orden ejecutiva.",  
  "text": "El presidente Trump firmó la orden ejecutiva este martes  
desde la sede de la Agencia de Protección Ambiental, rodeado de un  
grupo de mineros de carbón. Rodeado por un grupo de mineros de  
[...]"}  
}
```

Dataset XLSum - Ojo a la licencia

XL-Sum license:

Contents of XL-Sum repository are restricted to only **non-commercial research purposes** under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). Copyright of the dataset contents belongs to the original copyright holders.

Dataset Billsum

- Dataset de resúmenes abstractos de proyectos de leyes del estado de California.
- <https://github.com/FiscalNote/BillSum>
- Ejemplo de registro:

summary	text	title
Bureau of Land Management Foundation Act (Sec. 3) This bill establishes the Bureau of Land Management Foundation as a charitable, nonprofit organization to encourage, accept, obtain, administer, and use private gifts of money, devises, and bequests of real and personal property for the benefit of, or in connection with, the activities and services of the Bureau of Land Management (BLM). The foundation shall conduct and encourage programs and activities that support: educational, technical, scientific, and other assistance or activities to support the management of BLM lands with regard to wild free-roaming horses and burros, fish and wildlife and their habitats; National Conservation Lands, and recreation, cultural, and historic resources; and activities that support the reclamation and remediation of specified abandoned mine lands, specified orphaned oil and gas well sites, or public lands impacted by development connected to mineral exploration and development activities. [...]	SECTION 1. SHORT TITLE. This Act may be cited as the "Bureau of Land Management Foundation Act". SEC. 2. DEFINITIONS. In this Act: (1) Board--The term "Board" means the Board of Directors of the Foundation. (2) BLM--The term "BLM" means the Bureau of Land Management. (3) Chairman--The term "Chairman" means the Chairman of the Board. (4) Director--The term "Director" means an individual member of the Board. (5) Foundation--The term "Foundation" means the Bureau of Land Management Foundation established by this Act. (6) Secretary--The term "Secretary" means the Secretary of the Interior. (7) National conservation lands--The term "National Conservation Lands" means the system of lands established by section 2002 of the Omnibus Public Lands Management Act of 2009 (16 U.S.C. 7202). (8) Wild free-roaming horses and burros--The term "wild free-roaming horses and burros" [...]	Bureau of Land Management Foundation Act

Casos de uso: Clasificación - Modelo RoBERTalex

- Transformer pre-entrenado en español para el sector legal, open-source.
- Basado en el modelo base RoBERTa y pre-entrando con el **Spanish Legal Domain Corpora**, con un total de 8.9GB de texto.
- Listo para usar para la tarea **Fill Mask**.
- Para otras tareas, como **Question Answering, Text Classification, o Named Entity Recognition**, es obligatorio hacer un fine-tuning.
- <https://arxiv.org/abs/1907.11692>
- <https://huggingface.co/PlanTL-GOB-ES/RoBERTalex>

Casos de uso: Clasificación - Modelo FinBERT

- Model pre-entrenado en inglés para el sector financiero, open-source.
- Entrenado en datos financieros con los siguientes datasets:
 - **TRC-2**: 1.8M noticias financieras publicadas por Reuters entre 2008 y 2010.
 - **Financial PhraseBank**: dataset de noticias financieras con análisis de sentimiento. 4845 frases en inglés seleccionadas aleatoriamente de noticias financieras extraídas de la base de datos LexisNexis.
 - **FiQA Sentiment**: dataset QA creado para el congreso WWW '18.
- <https://arxiv.org/abs/1908.10063>

Casos de uso: Traducción

- Familia de modelos T5:
 - Modelo transformer encoder-decoder
 - Traducción de documentos financieros y de otro tipo
 - Soporta versiones en español (mT5)
 - Se puede realizar fine-tuning para adaptarlo a cualquier lenguaje.
 - <https://arxiv.org/abs/1910.10683>
- Familia de modelos Opus-MT (Universidad de Helsinki):
 - Entrenados usando el framework MarianMT (C++)
 - Arquitectura basada en transformers.
 - Entrenados con el dataset Opus y el tokenizador SentencePiece.
 - Los modelos Opus se han convertido a pytorch con la librería `transformers` de Hugging Face.
 - <https://arxiv.org/abs/2212.01936>

1. Búsqueda y síntesis de documentos financieros

- Empresas financieras dedican muchos recursos al **resumen de información y de documentos** internos.
- Los transformers pueden ayudar a encontrar información en contratos (pólizas, informes de créditos, préstamos, documentos regulatorios) y otros documentos PDF no estructurados. Ejemplo:
 - Prompt: “Haz un resumen de los requisitos regulatorios del banco X”.

2. Asistentes virtuales mejorados

- Clientes financieros necesitan encontrar respuestas a un **problema específico que es único y no está configurado en un chatbot AI existente** o disponible en las librerías de conocimiento que usa el chatbot.
 - Ejemplo: ayudar a un cliente en una transacción fraudulenta. Esta tipo de información no es trivial encontrarla en chatbots tradicionales ó librerías de conocimiento.
- Los *transformers* pueden ayudar a encontrar la información precisa.
- Los *transformers* permiten encontrar respuestas en grandes corpus de datos, los pueden resumir, y asistir a agentes humanos ó virtuales. Estas capacidades ayudan a mejorar las experiencias de servicio al cliente.

3. Investigación en mercados de capitales

- Para interpretar mercados y riesgos globales, las empresas de inversión deben **analizar todo tipo de documentos, transcripciones, informes y datos complejos** de las empresas en múltiples formatos, y consultar los datos de manera rápida.
- En los mercados de capitales, las herramientas de inteligencia artificial pueden servir como **asistentes de investigación para los analistas de inversiones**. Dichos asistentes ayudan a examinar millones de eventos (por ejemplo, resultados empresariales), documentos (por ejemplo, 10K/10Q), informes macroeconómicos, presentaciones regulatorias y otras fuentes, e identificar y resumir información clave de manera rápida.

4. Consultas de cambios regulatorios

- En la industria de servicios financieros, cada año surgen nuevas regulaciones a nivel mundial, mientras que las reglas existentes cambian con frecuencia, lo que requiere una gran cantidad de trabajo manual o repetitivo para interpretar los nuevos requisitos y garantizar el cumplimiento. Los desarrolladores **deben comprender rápidamente el cambio regulatorio que les requiera cambiar el código**, implementarlo y documentarlo:
 - Por ejemplo, los desarrolladores deben hacer cambios para cumplir con Basilea III que incluye miles de páginas de documentos. Con IA generativa, se podría resumir un área relevante de Basilea III para ayudar a un desarrollador a comprender el contexto, identificar las partes del marco que requieren cambios en el código y verificar el código con un repositorio de codificación de Basilea III.
- Los *transformers* pueden dar contexto a los desarrolladores sobre los cambios regulatorios, y pueden sugerir **cambios de código automáticamente**, con verificación humana, y generar la documentación.

5. Recomendaciones financieras personalizadas

- El Machine Learning (ML) tradicional es óptimo para estimar ventas ó realizar regresiones ó realizar ofertas de marketing según una serie de parámetros predeterminados
 - Ejemplo: crear emails de marketing ó mensajes a móviles con recomendaciones financieras específicas (aunque puede consumir recursos).
- Los *transformers* pueden ayudar en el proceso creativo de **desarrollar mensajería personalizada** usando lenguaje natural. Puede ayudar a mejorar la experiencia de cliente, retención y ventas.

Fin Sesión II

Pre-training. Modelos LLM primera generación

Rafael Sánchez, Ph.D.
contacto@rafaelsanchez.dev